Generative Specification: A Discipline of Derivability for the Stateless Reader

Author: Juan Carlos Ghiringhelli (PragmaWorks) **Version:** 4.0 · **Date:** June 2026 · **Status:** Preprint

> ***This is the white paper*** *— the focused, publication-length statement of the methodology, intended for Zenodo and arXiv. It is derived from the* ***Compendium*** *— the canonical, ~115-page source that carries the full evidence, the failure-mode catalog, the practitioner protocol, the formal-disciplines treatment, and the biological-isomorphism frontier. Every claim and number here is traceable there, and the experiment evidence is committed in the public repository. Where this paper compresses, the Compendium expands; both are kept consistent.*
>
> ***Full story and verification (all public).*** *The Compendium and six of the seven controlled experiments —* ***AX, BX, CX, DX, EX, KX, RX*** *— live in the public repository* `github.com/jghiringhelli/generative-specification` *(Compendium at* `docs/white-paper/GenerativeSpecification_Compendium.md` *; experiments, with per-run JSON evidence, under* `experiments/` *). The formal-tier experiment* ***ALX*** *lives in the public Loom repository at* `github.com/jghiringhelli/loom/tree/main/experiments/alx` *. The* `pragmaworks` *reference project is at* `github.com/jghiringhelli/pragmaworks` *.*

Abstract

The dominant failure mode of AI-assisted software development is **architectural drift**: an AI agent, capable of generating a system in a single session, resolves a thousand implicit decisions — units, field ordering, layer ownership, error semantics — silently, drawing on everything it has read about how such systems are *usually* built. Each decision is locally reasonable; in aggregate, across session, team, and service boundaries, they diverge. The cause is structural, not a model defect: the AI is a **stateless reader** that begins each session with none of the memory, shared context, or ability to ask that a human colleague uses to compensate for an underspecified system.

Generative Specification (GS) is the discipline that removes the freedom to leave architectural intent implicit: it requires a specification from which a stateless reader can derive correct output unaided. The discipline is operationalized as **seven properties** (Self-describing, Bounded, Verifiable, Defended, Auditable, Composable, Executable), each named for a failure mode observed in production and scored on a 14-point rubric. Its economic consequence is **cost inversion**: when regeneration from a specification is near-free, the specification — not the code — becomes the scarce, load-bearing artifact. We also propose a theoretical home for the discipline — the **pragmatic tier** of the semiotic hierarchy (Morris, 1938), the relation of signs to an interpreter that carries no context, a tier prior programming disciplines left vacant. We offer that placement as a conceptual contribution, distinct from and not required by the empirical results: the discipline stands on what it measurably produces, whatever one calls it.

We give the positive premise for *why* externalizing intent succeeds — **the bridge**: every structural discipline (naming, SOLID, domain language, contracts) is a bidirectional bridge between human conceptual language and executable code, and the transformer is the first reader trained on both banks. The bridge is asymmetric: the model is far stronger on human meaning than on exact code, so encoding intent in human-conceptual terms routes the hard half of the problem through the model's strong half. We connect this to retrieval economics: an authored specification is a member of the **compact-knowledge-graph** family, navigated rather than re-derived.

The empirical program is layered so that independent experiments establish distinct, individually-checkable claims. The discipline makes AI-derived software **measurably better-structured** (AX: 3/14 → 14/14 on the rubric across eight pre-registered conditions, 109 runner-verified tests, plus a ninth construction-invariance condition in which a tool-generated harness matched the best hand-built one); **independently reproducible** (RX: 104 passing tests regenerated from a committed specification by any third party with an API key); **deployable and operable in production** (EX: a full development→staging→production cycle on a live runtime — 13/13 behavioral probes carrying

1,013 assertions, 6/6 service-level-objective gates); and **cheaper to retrieve from** (KX: routed navigation beats both everything-in-context and code-search on accuracy *and* cost — macro-F1 0.808 vs 0.611 vs 0.431 at 79k vs 100k vs 234k tokens per query). Two further experiments extend the claim: GS is **derivable at the formal, machine-checkable tier** (ALX: a compiler derived from its own specification, 386/386 acceptance tests) and **transfers to practitioners in a single session** (DX: 58 developers, 116 submissions, 83 analyzable — reported with honest qualification). Six production projects supply the earlier discovery evidence; the controlled experiments carry the confirmatory weight.

---

1. Introduction: The Stateless Reader

On September 23, 1999, the Mars Climate Orbiter completed a nine-month, 416-million-mile crossing and arrived within 26 kilometers of its intended trajectory — then entered the Martian atmosphere at the wrong angle and was destroyed in 57 seconds. The cause was not a bug in any module. One team reported thruster impulse in pound-force seconds; the flight computer expected newton-seconds. Both components were internally correct, and *no test caught it — the code compiled cleanly.* The failure lived only at the boundary: the seam where two internally-coherent systems had to agree on a shared language they were never required to write down. The contract had been **assumed, not written**. It cost $327.6 million and two years to produce.

Today an AI assistant produces an equivalent interface in thirty seconds. The code compiles, types check, unit tests pass — and embedded in the output is a set of implicit assumptions the model resolved silently. The Orbiter problem did not go away; the velocity of producing it increased by orders of magnitude.

This is not an argument against AI-assisted development. It is an argument for what it requires. The Jacquard loom did not eliminate weaving craft — it relocated it upstream, into the card, where the pattern could be expressed once and executed at machine speed. Software construction is undergoing the same relocation. An AI agent with command-line access can read a codebase, write tests, run migrations, and commit, all in one session, starting from nothing. The executor is extraordinarily capable. The open question is what governs it across the session boundaries it cannot see, the team context it was never given, and the decisions made in conversations it was not part of.

The answer is a specification precise enough that a **stateless reader** — one with no memory of your intentions, no shared context, no ability to ask a clarifying question — can derive correct output from it alone. That discipline is what this work defines.

This work requires an AI agent with direct CLI access — not a chat assistant, but an agent that can read and write files, run tests, commit, and execute commands (Claude Code, Cursor, and agentic IDE modes qualify). The methodology does not apply to chat-based interfaces.

Contributions. (1) The **derivability obligation** — naming implicit-context removal as the defining structural constraint of AI-assisted development. (2) **A proposed theoretical placement** — situating the discipline in the pragmatic tier of the syntactic/semantic/pragmatic tripartition, offered as a conceptual lens distinct from (and not load-bearing for) the empirical claims. (3) The **seven-property rubric** — a teachable, scored instrument validated across production projects and 83 practitioner submissions. (4) **Phase collapse** — specification, implementation, and verification collapsing into a single derivation step when the spec is complete and the executor is capable. (5) **The bridge and its asymmetry** — the positive premise for why externalized intent is derivable, and why the load moves to the model's stronger bank. (6) **Cost inversion and the token economics of authored structure** — tokens-per-correct-output, not tokens-generated, as the binding metric. (7) A **layered validation program** (AX/BX/DX plus EX/KX/ALX/RX) in which independent experiments close distinct sources of circularity.

---

2. A Discipline of Removal, and Its Theoretical Place

A paradigm, in Robert C. Martin's precise sense, is defined by what it *removes* from programmer freedom. Structured programming removed `goto`. Object-oriented programming removed unconstrained access to internal data.

Functional programming removed unrestricted reassignment. **Generative Specification removes the freedom to leave architectural intent implicit.**

This removal differs in kind from its predecessors. The earlier paradigms constrained freedoms for human readers who could still compensate for gaps through memory, collaboration, and institutional context. GS's removal is *absolute for its reader*: a stateless executor that begins each session with none of those recovery mechanisms. What prior paradigms made inconvenient, GS makes structurally absent — because the reader that would have compensated does not exist.

The semiotic tripartition — **syntactics, semantics, pragmatics** (Morris, 1938) — locates the discipline. Syntactic disciplines constrain the *form* of artifacts (what constructs are permitted, what dependency directions are allowed). Semantic disciplines constrain *meaning* (types, schemas, contracts). The **pragmatic** tier is the relation of signs to their interpreter in a context of use. GS occupies the pragmatic tier because it governs derivability for a reader who carries *no* interpretive context — the tier prior disciplines left vacant. Applying Morris's settled vocabulary to classify programming discipline is this work's proposal; the claim rests on the structural observation that no prior discipline stated the obligation to make the lifecycle layer derivable for a contextless executor. We present this placement as the work's conceptual contribution, not as a result to be defended empirically — the discipline and its measured effects in §5 stand whether or not a reader accepts the tier framing. The word *paradigm* is used here only in Martin's narrow, technical sense (a discipline of removal); we make no Kuhnian claim about the field, which the adoption data has not yet earned.

---

3. The Seven Properties

The discipline is operationalized as seven properties, each named for a class of failure observed in production, each scored 0–2 for a 14-point total. The rubric is the teachable spine of the method: a project is graded the way an AI reads it. Each property below is anchored to a **public, verifiable exemplar** — drawn from the open experiments or the public `pragmaworks` reference project, so a reader can inspect the evidence directly.

#	Property	What it removes	Public exemplar
1	**Self-describing**	Hidden purpose; the reader must infer what the system is	`pragmaworks` `CLAUDE.md` — a navigation root where every document location announces its domain (screaming architecture)
2	**Bounded**	Unbounded surface and context; the reader must scan everything	The DX study (`experiments/dx`) machine-counts bounded violations per submission — e.g. direct DB access bypassing the repository layer — across all 83 analyzable branches
3	**Verifiable**	Unchecked correctness; "it compiles" mistaken for "it works"	AX (`experiments/ax`): a Stryker mutation gate drove the mutation score from 58.6% to 93.1% MSI — proving *detection*, not merely line execution
4	**Defended**	Advisory rules the model treats as optional	AX (`experiments/ax`): Defended moved 0/2 → 2/2 only once gates were emitted as fenced file templates (the "First Response Requirements") — structural enforcement, measured
5	**Auditable**	Lost rationale; intentional decisions look like debt	`pragmaworks` ADR library (`docs/adrs/0001…0006`, each with context/decision/consequences) plus conventional-commit history

6	**Composable**	Tangled coupling that cannot recombine	AX (`experiments/ax`): interface-based dependency injection — the GS contribution over expert prompting — lets a stateless reader work a unit in isolation
7	**Executable**	Specifications that are never run against reality	EX (`experiments/ex`): Hurl probes + `slo-ramp-summary.json` — behavioral contracts run against a live runtime, not assumed from compilation

These properties are not independent virtues; they partition by **functional role** (§4.3). Two — Self-describing and Bounded — carry disproportionate weight, because a bounded, self-describing specification activates the model's relevant domain knowledge (a schema-fit effect: Bransford & Johnson, 1972) rather than its full prior distribution.

The Verifiable and Defended properties are not new instruments; they are the obligation to *apply* the standard quality toolchain and gate on it. The verification layer is assembled from type checkers, linters (ESLint), test-coverage and cyclomatic-complexity gates, mutation testing (Stryker), dependency-vulnerability scans (`npm audit`), and quality-gate platforms such as SonarQube. GS does not reinvent these — it specifies which gates apply where and makes passing them the definition of *done* for a stateless executor that otherwise reports success on byte-write, not on correctness. Those same tools, being rubric-independent, double as external corroboration of the GS scores (§6).

The properties also generate a **diagnostic catalog**: their observable violations resolve into twenty-nine named pathologies (Architectural Drift, Session Amnesia, Implicit Contract Syndrome, and so on), each mapping to one or more absent properties and to the tier of the lifecycle cascade that structurally repairs it. The catalog is the rubric made actionable; it is developed in full in the Compendium.

---

4. Why It Works

4.1 The Bridge

The negative premise of GS is the stateless reader (the problem) and the navigational structure that bounds its context (a mechanism). The **positive** premise — why externalizing intent into structure actually *produces correct derivation* — is the bridge.

Every structural discipline is a **bridge between human conceptual language and executable code**: intentional naming, the specification, SOLID, domain-driven ubiquitous language, type-driven design, design by contract. Each encodes human meaning in a form the machine can act on, and machine behavior in a form the human can verify. These disciplines were built to carry intent across that gap for the *next human reader*. The transformer is the **first reader trained on both banks** — the corpus of human language and the corpus of code — and therefore the first that can cross the bridge in both directions: read intent encoded as structure, and emit code that encodes intent. GS works because it makes building and maintaining that bridge the primary act of development.

The bridge is **asymmetric**, and this is the sharper claim. The training corpus is overwhelmingly natural language; code is a small, exact, fragmented slice — every language its own precise rules, one wrong token breaks it. The model is therefore far stronger on human-conceptual meaning than on exact code. The leverage of the disciplines is not merely that they connect both banks: it is that **encoding intent in human-conceptual terms routes the hard half of the problem (exact code) through the model's strong half (human meaning it is fluent in)**. The discipline moves load from the model's weak bank to its strong bank.

4.2 Cost Inversion

In traditional development, implementation accumulates a sunk cost; when a specification conflicts with a built system, the rational response is to amend the specification, because the code is load-bearing and the specification is not. GS inverts this. When regeneration is near-free, implementation carries no sunk cost: fix the specification and

regenerate. The specification is *not* reliably recoverable from code — decisions, alternatives considered, and accumulated rationale resist reconstruction. Code becomes an implementation residue. **The scarce resource is no longer the ability to write code; it is the ability to specify correctly.**

A directional model formalizes this: **I ∝ (1−S)/S**, where *S* is specification completeness — the fraction of the output space the specification closes — and *I* is the expected number of correction iterations. It is a *mental model, not a formal result*: no units, no proportionality constant, no magnitude prediction. What it communicates is direction — each freedom the specification leaves unclosed is an additional correction cycle, and the cost rises sharply as *S* falls toward zero. The AX series gives cross-condition directional support (3/14 → 14/14 across eight conditions as *S* rises); DX will supply the first cross-practitioner correlation test. Full treatment in the Compendium (§9.4).

4.3 Token Economics and the Discipline-Role Taxonomy

The one substantive objection GS meets in the field is token expenditure: writing the specification, the cascade documents, and the tests, then regenerating, all appear token-expensive. The honest reconciliation has two parts. First, the binding metric is **tokens-per-correct-output**, not tokens-generated: without authored structure, a session burns tokens on discarded wrong code, re-explained context every cold start, and drift repair. Second — and observed directly in the field — under inversion of control the *absolute* spend rises because the practitioner advances faster; **what is purchased is time, not tokens.** Both are true and not in tension. The per-correct-output economy is a *reasoned argument*, not an end-to-end measurement: KX (§5.2) measures the retrieval component directly — authored structure navigated rather than re-derived — but the full-session figure is not measured here. A widely-circulated ~70% token-reduction claim was conceded unproven in the field (§5.3) and is deliberately *not* asserted; the defensible claim is the KX-measured retrieval economy plus the reasoned downstream argument.

The mechanism is retrieval economics. An authored specification is a member of the **compact-knowledge-graph (CKG)** family — a small, enumerable, closed-vocabulary structure the reader navigates instead of re-deriving from prose at every query. Yarmoluk and McCreary (2026) benchmark this directly for knowledge retrieval: a pre-authored CKG costs ≈11× fewer tokens at ≈3.8× higher accuracy than chunked-prose RAG or query-time graph extraction, concluding that "when expert structure is available, the dynamic extraction step is wasted computation." The GS navigation tree generalizes the CKG's single prerequisite relation to the executor's full operating path: navigation across cascade documents, dependency direction, applicable disciplines, and inviolable constraints.

This frames a GS session as **retrieve → generate → verify**. The read side is a retrieval problem, attacked from both ends — authoring the structure (the navigation tree) *and* shaping the code to be retrievable (the disciplines), with a code-search engine as the traversal. The harness is the categorically distinct *verify* step, which checks output against the specification. GS is therefore retrieval-augmented *and verified* generation, with the retrieval **authored rather than inferred**.

This sorts the seven properties by **functional role**:

- **Verify** (secure correctness): TDD, BDD, design by contract, type-driven design, and the enforcement gates → Verifiable, Defended, Executable. Enforced by hooks, CI, and the harness.
- **Retrieve** (the bridge), in three sub-roles:
  - *Legibility* — read the **what**: naming, SOLID interfaces, hexagonal layering, ubiquitous language → Self-describing, Composable. Enforced by the navigation tree and a structural code index.
  - *Bounding* — leave **less** to read: DRY/deduplication, dead-code elimination, YAGNI, small units → Bounded. A second, independent lever on token cost (a smaller surface, not just better navigation), enforced by automated deduplication and dead-code detection over the dependency graph.
  - *Decision-memory* — read the **why**: ADRs, conventional commits, engineering decision records → Auditable. Enforced by the cascade documents.

A discipline may serve more than one role; the grouping is by dominant function. Naming the role explains why the seven properties partition as they do — and why a codebase strong on verification yet weak on legibility and

bounding still reads expensively.

---

5. Evidence

The evidence falls into two categories of different epistemic weight. The **six production projects** are the substrate from which the methodology was *derived* — real systems built or refactored under increasingly rigorous discipline, surfacing a failure mode each and producing a corrective property. They carry the weight of discovery; built under early, still-maturing versions of the discipline, they are not the best demonstrations of it. The **seven controlled experiments** are the *testing* phase, conducted after the methodology stabilized, with prospectively committed criteria designed to falsify rather than illustrate. The controlled results carry the confirmatory weight.

5.1 Production Projects (Discovery)

Project	Setting	Property it surfaced
SafetyCorePro	Takeover of an AI-built system	Defended (operational enforcement was absent)
Invellum	Brownfield refactor	Bounded (unbounded surface)
Conclave	Complex greenfield	Composable (layer discipline at scale)
BRAD	Legal-intelligence extension	Self-describing (technique transport by naming)
Shattered Stars	Language/engine migration	Auditable (decision provenance lost across sessions)
Regulated Multi-Layer Data Platform	Staging + production governance	Executable (contracts proven against running tiers)

(ForgeCraft, the methodology's own tooling, is treated separately as the self-application case, not as a proof project.)

5.2 Controlled Experiments (Testing)

AX — the discipline produces measurably better-structured output (multi-agent adversarial study). Eight pre-registered conditions on the RealWorld Conduit benchmark, single practitioner, blind external rubric. The naive condition scored 3/14 (its test suites failed to compile); structured conditions reached the ceiling, with **two iterations (Treatment-v3, Treatment-v5) at 14/14**, 109 runner-verified tests (106 passing on independent re-run) against a live PostgreSQL database, and a documented **non-monotonic** path (a regression at v4 to 11/14, recovered at v5). A ninth condition tested **construction invariance**: a *tool-generated* harness (zero hand-tuning) reached the same ceiling as the best hand-built arm — 12-of-12 blind audit, 14/14 expanded scale, 11/11 live use-case probes. The structural advantage does not depend on expert hand-authorship.

DX — the discipline transfers to practitioners in a single session (58-developer study, April 2026, reported with honest qualification). Two greenfield projects in sequence; 58 developers produced two submissions each (116 total; 83 analyzable, the rest empty or incomplete and concentrated in the second, post-reveal session). In the pre-reveal pure-tool comparison (Vaquita), free prompting produced three times as many perfect-score outputs as the scaffolded condition (38% vs 13%, χ^2 $p < .05$; overall $p = .076$, Cohen's $d \approx 0.5$) — front-loading specification trades implementation momentum for structural discipline within a fixed time budget. Post-reveal, practitioners who *internalized* the methodology and applied it freely outperformed those constrained by an artifact generated before they understood the project (75% vs 63% perfect). The findings are reported honestly: a single-session exposure with an underpowered design and a temporal-mismatch confound that is a product issue, not a methodology one.

EX — the result is deployable and operable in production. A Conduit backend specified, generated, verified, and deployed to a live cloud runtime (April 2026), the harness driving the full development→staging→production cycle. **Level 2:** 13/13 behavioral (Hurl) probes carrying 1,013 assertions, derived from use-case acceptance criteria, against the deployed service. **Level 3:** 3/3 environment-governance probes (configuration, secret hygiene, reachability). **Level 4:** 6/6 service-level-objective gates under load (aggregate p95 350 ms, p99 720 ms, error rate 0.04%). Fifteen defects were surfaced by failing probes and fixed before the cycle could close. This is the Executable property earned against a running system, not assumed from compilation.

KX — authored structure is cheaper to retrieve from (knowledge-retrieval replication). The CKG benchmark method run on a GS software harness across three conditions, each a fresh agent session per query: a routed navigation tree (CKG analog) reached macro-F1 **0.808 at 78.6k tokens/query**, beating both everything-in-context (RAG-dump: 0.611 at 100k) and code-search-at-query-time (no-structure: 0.431 at 233k) on accuracy *and* cost — 5.5× cheaper per query than the monolith. The architectural divergence replicated: aggregate-enumeration queries scored 0.909 (routed) vs 0.006 (no-structure), mirroring the original benchmark's 0.964 vs 0.054. Token sanitation is measurable, not aspirational; the absence of structure is the most expensive condition.

ALX — derivability holds at the formal, machine-checkable tier. A complete compiler for the Loom language was derived from its own formal specification alone (`cargo check` passed on first emission). The contribution is the correction curve from specification gaps to **S_realized = 1.0** (0.000 → 0.339 → 0.642 → 0.781 → 0.900 → 1.000 across six phases, terminating at **386/386** acceptance tests). Each correction was a *specification* improvement, not a code patch — enumerating exactly which classes of detail a formal specification must carry to be machine-derivable. This is spec derivability demonstrated at the machine-checkable layer above natural language, where conformance is `cargo test` , not a rubric. Evidence: `experiments/alx` in the public Loom repository (spec, derived source, and correction log committed).

RX — the result is independently reproducible. From a single committed specification, any reader with Docker, Node, and an API key regenerates **104 passing tests across seven suites** (recorded run March 2026), with committed evidence (`jest-output.json` , `score.json` , build log). The result does not rest on the author's word; it is a button a third party can press.

BX — the rubric measures something independent of its author (blind cross-validation). Three community implementations never exposed to GS were scored on the rubric; the rankings were congruent with independent external metrics (CVE count, test count, type-health), establishing that the rubric measures something that exists independent of its author.

5.3 Field Corroboration (Observational)

A four-day GS workshop delivered to a paying eight-developer cohort at an insurance brokerage (June 2026) is reported as *observational* evidence, distinct from the controlled program (no control, no blind scoring, single self-selected cohort). It cannot test a hypothesis; it shows whether the controlled findings recur when GS meets a real team on its own code. On the greenfield day, every project demonstrated produced a running artifact within a single morning, and residual defects traced to *under-specification* or a missing asset rather than to the method — one participant diagnosing his own failed build without prompting: "*it is absolutely my fault, because I did not specify it correctly.*" By the brownfield day every participant had mapped GS onto a production system, and one instituted a process change of his own accord, making an architecture decision record a merge prerequisite. The token objection recurred with a consistent trajectory — caution, an acute mid-workshop episode of exhausted budgets, then qualified willingness to adopt — corroborating the time-not-tokens reconciliation of §4.3.

---

6. Threats to Validity

The most serious risk is **guidance circularity**: GS guided the implementations and supplied the rubric. The validation program is layered to close it. BX applies the rubric to implementations it never guided and finds the rankings

congruent with rubric-independent metrics — CVE count (`npm audit`), test count, type-health (`tsc`), and cyclomatic complexity. RX makes the executable result reproducible from archived artifacts by any third party. DX breaks the guidance-plus-measurement loop entirely with human participants and a blind evaluator. KX's structural-query ground truth derives from the same structure the navigation tree reads (the benchmark's own caveat), so its claim is bounded: *explicit structure beats inferred structure on structural queries*, not general superiority. Results are predominantly single-model (Claude); the AX series is single-practitioner and directional (eight pre-registered conditions, non-monotonic, with a ninth testing construction invariance); DX is single-session and underpowered, with formal (powered) inference deferred to a follow-up. The inverse-effort relation between specification completeness and correction cost is offered as a directional model, not a formal result. The six production projects are author-evaluated discovery substrate, not controlled evidence.

---

7. Implications for Practice

The specification precedes the code. The architectural constitution, structural diagrams, schema definitions, and at least a skeleton decision record must exist before the first agent session. This is not new; it was optional when the cost of skipping it was paid by a human who could compensate with memory. That compensation is unavailable to a stateless executor.

Specification-first, iterative delivery. GS is not a third methodology beside waterfall and agile. The specification layer runs waterfall — the grammar is written first; the delivery layer runs agile — each session produces atomic, tested, deployable commits. Waterfall's rigid front-loading dissolves because the constitution is a living document revised through commit discipline; agile's structural drift dissolves because the specification gates every session. Scope may be bounded: a complete specification of a minimum viable slice is still complete.

The industrial threshold. Three constraints historically prevented sustained formal discipline — learning (no career is long enough), maintenance (discipline erodes under deadline), and transfer (knowledge lived in people). A tireless executor that reads the specification before every session makes all three irrelevant. The practitioner's role shifts from implementation artisan to specification architect. The complexity moved upstream, into the card.

---

8. Conclusion

Architectural drift at generation speed is the anomaly that documentation-based convention cannot structurally prevent. The reconstitution is the specification becoming the primary artifact, with code as derived output, governed for a reader that carries no context of its own. The discipline is replicable (RX), measurable (the rubric, KX), and demonstrable in production (EX) and at the formal tier (ALX); it transfers to practitioners in a single session (DX, with honest qualification) and recurs in the field. The specification is the mold. The AI is the foundry. The scarce resource — the one that does not regenerate for free — is the judgment to specify correctly.

References (selected)

Full bibliography, glossary, the twenty-nine-pathology catalog, the practitioner protocol, the formal-disciplines treatment, and the biological-isomorphism frontier are in the **Compendium**.

- Morris, C. W. (1938). *Foundations of the Theory of Signs*.
- Martin, R. C. *Clean Architecture* — paradigm as constraint.
- Bransford, J. D., & Johnson, M. K. (1972). Contextual prerequisites for understanding. *JVLVB*.
- Yarmoluk, D., & McCreary, D. (2026). *Benchmarking Knowledge Retrieval Architectures: RAG, GraphRAG, and Compact Knowledge Graphs.* v0.6.2 preprint.
- Gordon (2024, ACM Onward!). *The Linguistics of Programming*.
- NASA (1999). *Mars Climate Orbiter Mishap Investigation Board Final Report*.

- Experiment evidence (AX/DX/EX/KX/ALX/RX): `experiments/` in the public repository, with committed per-run JSON.